MODULE 10 · DOMAIN 2 - CONFIGURATION MANAGEMENT AND IAC

Infrastructure as Code at Scale

StackSets across accounts and Regions, modules for reuse, Service Catalog for self-service, and Guard with Hooks for guardrails.

DOMAIN 2 of 6 EXAM WEIGHT 17% MODULE 10 of 34

admin@sthithapragna.com

Four problems of scale, four tools

PROBLEM	TOOL
The same template in many accounts and Regions	CloudFormation StackSets
The same snippet repeated in many templates	CloudFormation modules, or CDK constructs
Teams need approved resources without broad IAM power	AWS Service Catalog
Non-compliant resources must be stopped before they exist	CloudFormation Guard and Hooks

ON THE EXAM

Distribution, reuse, self-service, guardrails. Name the problem and the tool follows.

Self-managed or service-managed StackSets?

SERVICE-MANAGED

Uses AWS Organizations

- No roles to create in each account
- Target the organisation or OUs
- Automatic deployment to new accounts
- Management or delegated admin account runs it

SELF-MANAGED

You create the roles

- An administration role, and an execution role per target
- Target individual account IDs
- No automatic deployment
- Works outside an organisation

THE TIEBREAKER

Inside an organisation, targeting OUs, including future accounts: service-managed. Accounts outside it: self-managed.

Operation preferences: controlling the blast radius

SETTING	WHAT IT CONTROLS
Maximum concurrent accounts	How many accounts per Region are worked on at once
Failure tolerance	How many accounts per Region may fail before stopping
Region concurrency	Sequential, one Region at a time, or parallel
Region order	Which Region goes first when sequential
Concurrency mode	Strict: slows down as failures occur. Soft: keeps its pace

ON THE EXAM

Exceed the failure tolerance in a Region, and the operation stops there and skips every later Region.

The arithmetic of a safe rollout

- Defaults are cautious: one account at a time, and zero failures tolerated.
- In strict mode, maximum concurrency can be at most **failure tolerance plus one**.
- Tolerance 2, concurrency 3: at worst three accounts fail before it stops.
- Operation preferences do **not** apply to automatic deployments to new accounts.

> ON THE EXAM
>
> 'Stop quickly if more than two accounts fail, one Region at a time': tolerance 2, sequential, with a Region order.

Service-managed details that get examined

- Targeting an OU includes its child OUs. Account filters can narrow or widen the targets.
- The **management account never receives stacks**, even if it sits in a targeted OU.
- When an account leaves a target OU, its stacks are deleted or retained, as you choose.
- **StackSet dependencies**, up to ten, order automatic deployments between StackSets.

> ON THE EXAM
>
> A baseline never appears in the management account: expected. Deploy there separately, with a normal stack.

When StackSet instances go wrong

- **Parameter overrides** give one account or Region different values from the same StackSet.
- Names that must be unique, such as S3 buckets or IAM roles, collide across Regions or accounts.
- Include the account ID and Region in such names, or let CloudFormation generate them.
- Drift detection works across a whole StackSet, instance by instance.

> ON THE EXAM
>
> StackSet fails in the second Region only, 'already exists': a globally named resource is colliding.

A module, or a nested stack?

CLOUDFORMATION MODULE

Expands into your template

- Registered in the registry, type ending ::MODULE
- Its resources join the parent stack
- Versioned, shareable building block
- No separate stack to manage

NESTED STACK

A stack inside a stack

- A separate child stack, with its own events
- Resources belong to the child
- Referenced by template URL
- Its own limits and lifecycle

THE TIEBREAKER

Reusable, versioned snippet with no extra stacks: a module. A separately managed component: a nested stack.

AWS Service Catalog: approved self-service

- A **product** is a versioned template: CloudFormation, or Terraform.
- A **portfolio** groups products and grants them to users, groups or roles.
- Users launch **provisioned products**, choosing only the parameters you expose.
- Users need Service Catalog permissions, not permissions for the resources underneath.

> ON THE EXAM
>
> Teams must launch only approved, compliant configurations, without broad permissions: Service Catalog.

Service Catalog constraints

CONSTRAINT	WHAT IT DOES
Launch constraint	Names an IAM role that Service Catalog assumes to provision
Template constraint	Limits parameter values, such as allowed instance types
TagOptions	Applies approved tags, or lets users choose from approved values
Notification constraint	Sends stack events to an SNS topic
StackSet constraint	Lets a product deploy to several accounts and Regions

ON THE EXAM

The launch constraint is how least privilege works: the role creates the resources, not the user.

Sharing and updating a catalogue

- Portfolios can be shared with accounts, or with OUs through AWS Organizations.
- The receiving account imports the portfolio and grants it to its own users.
- New template versions become new product versions. Users update their provisioned products.
- A delegated administrator account can manage organisation sharing.

> ON THE EXAM
>
> One central catalogue for every account in the organisation: share the portfolio with the OU.

CloudFormation Guard: policy as code

- **Guard** is a rule language for checking JSON or YAML against your policies.
- It checks CloudFormation templates, and also Terraform plans and Kubernetes manifests.
- **cfn-guard validate** in a build fails the pipeline on a violation, before any deployment.
- **cfn-lint** is different: it checks template syntax and best practice, not your organisation's rules.

> ON THE EXAM
>
> The same Guard rules can run in three places: a developer's machine, the build, and a Hook at deploy time.

A Guard rule

```
let buckets = Resources.*[ Type == 'AWS::S3::Bucket' ]

rule buckets_are_versioned_and_private when %buckets !empty {
  %buckets.Properties.VersioningConfiguration.Status == 'Enabled'
  %buckets.Properties.PublicAccessBlockConfiguration.BlockPublicAcls == true
  %buckets.Properties.PublicAccessBlockConfiguration.RestrictPublicBuckets == true
}
```

1 Select every S3 bucket resource in the template.

3 Only evaluate the rule if there are buckets to check.

ON THE EXAM

Every clause must hold for every selected resource. One unversioned bucket fails the rule.

CloudFormation Hooks: guardrails at deploy time

- Four kinds: **Control Catalog** proactive controls, **Guard**, **Lambda**, and custom Hooks.
- Targets: resources, whole stacks, change sets, and Cloud Control API calls.
- Failure mode **FAIL** stops the operation; **WARN** lets it continue with a warning.
- Hooks are set per account and Region. Roll them out organisation-wide with StackSets.

> ON THE EXAM
>
> Since September 2025, Hooks can apply Control Tower's managed proactive controls, with no rules to write.

One policy, checked at every stage

01
Developer
cfn-lint and cfn-guard locally, in seconds.

02
Pull request
Guard in a CodeBuild check before merge.

03
Pipeline
Guard fails the build: nothing deploys.

04
Deploy time
A Hook in FAIL mode blocks what slipped through.

05
After deploy
AWS Config rules detect changes made later.

ON THE EXAM

Earlier stages give faster feedback. The Hook is the final gate that catches anything else.

Hook, SCP, or Config rule?

CLOUDFORMATION HOOK

Preventive, rich checks

- Sees full resource properties
- Only CloudFormation and Cloud Control operations
- Stops the stack operation
- Per account and Region

SCP OR CONFIG RULE

Broader, but different

- SCP: every API call, but only condition keys
- SCP: cannot inspect most resource properties
- Config: any change, but after the fact
- Config: detective, with optional remediation

THE TIEBREAKER

Check properties of resources deployed through CloudFormation, before creation: a Hook. Deny an API everywhere: an SCP.

A service-managed StackSet deploys a monitoring baseline to 60 accounts in four Regions. A recent update failed in many accounts at once before anyone noticed.

For future updates, the company wants Regions updated one at a time, starting with eu-west-1, and wants the update to stop as soon as more than two accounts fail in a Region.

Which operation preferences should the DevOps engineer set?

A. Parallel Region concurrency, with a failure tolerance of 2

B. Maximum concurrent accounts of 100%, with soft failure tolerance

C. Sequential Region concurrency with eu-west-1 first in the Region order, and a failure tolerance of 2

D. Soft failure tolerance with 60 maximum concurrent accounts

◆ PAUSE AND DECIDE

Which operation preferences should the DevOps engineer set?

A Parallel, tolerance 2

Right tolerance, but every Region at once. Fails 'one at a time'.

B 100% concurrency, soft

Everything at once and keeps going through failures: the incident, repeated.

C Sequential, eu-west-1 first, tolerance 2

One Region at a time in a chosen order, stopping after the third failure in a Region.

D Soft, 60 concurrent

Soft mode ignores failures when setting pace. The opposite of stopping quickly.

THE SIGNAL

One Region at a time: sequential. Starting with a named Region: Region order. Stop after N failures: tolerance N.

Developers frequently need PostgreSQL databases for new services. Each must be encrypted, have backups enabled, and use approved instance classes.

The security team will not give developers permission to create RDS resources directly. The company wants developers to self-serve compliant databases with the least operational overhead.

Which solution meets these requirements?

A Grant developers RDS permissions, and use an AWS Config rule to flag non-compliant databases

B A CodeBuild project that developers start, which creates the database with the AWS CLI

C Publish an approved template in S3 and let developers deploy it with CloudFormation

D A Service Catalog product in a portfolio shared with developers, with a launch constraint role and a template constraint on instance classes

◆ PAUSE AND DECIDE

Which solution meets these requirements?

A RDS permissions plus Config

Gives developers exactly the permission the security team refused, and only detects problems later.

B CodeBuild with the CLI

A home-built self-service system: parameters, validation and updates all to maintain.

C Template in S3

Deploying it still needs RDS permissions, and developers could edit the template.

D Service Catalog, launch and template constraints

The launch role creates the database; developers need only catalogue access and pick approved values.

THE SIGNAL

Self-service without the underlying permissions: Service Catalog with a launch constraint.

A security team has written a Guard Hook requiring encryption with a customer managed key on every DynamoDB table. Forty teams deploy with CloudFormation.

The security team is concerned that blocking immediately would break some teams' releases. It wants to see which stacks would fail before enforcing the rule.

What should the security team do?

A Activate the Hook in FAIL mode, filtered to a single test stack

B Replace the Hook with an AWS Config rule

C Activate the Hook in WARN mode, review its invocation results, then switch to FAIL mode

D Write an SCP denying dynamodb:CreateTable without customer managed key encryption

◆ PAUSE AND DECIDE

What should the security team do?

A FAIL on one test stack

Tells you nothing about the other forty teams' stacks.

B Config rule

Detects after creation. Gives up the preventive goal the Hook was written for.

C WARN, review, then FAIL

Every stack is evaluated and warned, nothing is blocked, and enforcement follows once teams have fixed things.

D SCP

Blocks immediately, the exact risk the team wants to avoid, and cannot inspect the property reliably.

THE SIGNAL

See the impact before enforcing: WARN mode first, invocation summaries, then FAIL. AWS documents this rollout.

> A Guard Hook in FAIL mode blocks non-compliant stacks at deploy time. Developers only learn of violations when a pipeline reaches its deploy stage, around 20 minutes after they push.
>
> The team wants developers to see the same violations within a minute or two of pushing, before deployment, with the least effort. The Hook must remain.

What should the DevOps engineer do?

A. Add an AWS Config rule that evaluates the same policy

B. Reduce the Hook's timeout so it fails faster

C. Switch the Hook to WARN mode so pipelines are not blocked

D. Run cfn-guard validate with the same rule files in the build and pull request checks, failing on violations

◆ PAUSE AND DECIDE

What should the DevOps engineer do?

A Config rule

Evaluates after deployment. Later feedback, not earlier.

B Shorter Hook timeout

The Hook is quick. The 20 minutes is the pipeline reaching it.

C WARN mode

Removes the protection, and developers still hear nothing until deploy.

D cfn-guard in build and pull request

Same rules, run minutes after a push. The Hook stays as the final gate.

THE SIGNAL

Same policy, earlier feedback: run Guard in the build. Keep the Hook for anything that bypasses the pipeline.

Module 10 in five lines

- Service-managed StackSets for OUs and future accounts; never the management account.
- Failure tolerance per Region stops the rollout; sequential Regions follow your order.
- Modules expand into templates; nested stacks are separate stacks.
- Service Catalog launch constraints give self-service without underlying permissions.
- Guard in the build for fast feedback; Hooks at deploy time; start Hooks in WARN.

ON THE EXAM

Next, module 11: the AWS CDK and AWS SAM.